MapStruct & DTOs Implementation Guide - CivilControlBackend

Overview

This document explains the complete implementation of MapStruct for object mapping and the DTO (Data Transfer Object) pattern in the CivilControl Backend system, providing clean separation between entities and API responses, automatic mapping, and type-safe transformations.

Architecture

Core Components

1. **MapStruct Mappers** - Auto-generated mapping interfaces
2. **DTOs** - Data Transfer Objects for API communication
3. **Entities** - JPA entities for database persistence
4. **Response DTOs** - Enriched DTOs with additional information
5. **Filter DTOs** - Query parameter objects for filtering

Configuration

Maven Configuration (pom.xml)

<properties> <org.mapstruct.version>1.5.5.Final</org.mapstruct.version>
<lombok.version>1.18.30</lombok.version> </properties> <dependencies> <!-- MapStruct -->
<dependency> <groupId>org.mapstruct</groupId> <artifactId>mapstruct</artifactId>
<version>${org.mapstruct.version}</version> </dependency> <!-- Lombok --> <dependency>
<groupId>org.projectlombok</groupId> <artifactId>lombok</artifactId>
<version>${lombok.version}</version> <optional>true</optional> </dependency>
</dependencies> <build> <plugins> <plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId> <configuration> <annotationProcessorPaths>
<path> <groupId>org.projectlombok</groupId> <artifactId>lombok</artifactId>
<version>${lombok.version}</version> </path> <path> <groupId>org.mapstruct</groupId>
<artifactId>mapstruct-processor</artifactId> <version>${org.mapstruct.version}</version>
</path> <path> <groupId>org.projectlombok</groupId>
<artifactId>lombok-mapstruct-binding</artifactId> <version>0.2.0</version> </path>
</annotationProcessorPaths> </configuration> </plugin> </plugins> </build>
Key Point: lombok-mapstruct-binding ensures Lombok and MapStruct work together correctly.

DTO Pattern

DTO Types

1. Input DTOs (Request)
- Receive data from clients
- Contain validation annotations
- Use Java Records for immutability

2. Response DTOs
- Send data to clients
- Include computed fields
- May aggregate data from multiple entities

3. Filter DTOs
- Query parameters for filtering/searching
- Optional fields for flexible queries

Example: Vehicle DTOs

1. VehicleDTO (Input)

public record VehicleDTO(@NotBlank(message = "{vehicle.licensePlate.required}") @Pattern(regexp = "^[A-Z]{3}\\d{3}$", message = "{vehicle.licensePlate.pattern}") String licensePlate, @NotNull(message = "{vehicle.brand.required}") String brand, @NotNull(message = "{vehicle.model.required}") String model, @NotNull(message = "{vehicle.year.required}") @Min(value = 2000, message = "{vehicle.year.min}") Integer year, @NotNull(message = "{vehicle.vehicleType.required}") VehicleType vehicleType, @NotNull(message = "{vehicle.fuelType.required}") FuelType fuelType, @NotNull(message = "{vehicle.projectAreaId.required}") Long projectAreaId, TruckEquipment truckEquipment, String observations) {}

Features:
- ✅ Immutable (Java Record)
- ✅ Validation annotations
- ✅ i18n message keys
- ✅ Clean, concise syntax

2. VehicleResponseDTO (Output)

public record VehicleResponseDTO(Long id, String licensePlate, String brand, String model, Integer year, VehicleType vehicleType, FuelType fuelType, TruckEquipment truckEquipment, String observations, // Related entity info Long projectAreaId, String projectAreaName, // Metadata Boolean active, LocalDateTime createdAt, LocalDateTime updatedAt) {}

Features:
- ✅ Includes entity relationships
- ✅ Includes metadata
- ✅ Read-only (no setters needed)

- ✅ Complete information for frontend

3. VehicleFilterDTO (Query Parameters)

public record VehicleFilterDTO(String licensePlate, String brand, String model, Integer minYear, Integer maxYear, VehicleType vehicleType, FuelType fuelType, Long projectAreaId, Boolean active) {}

Features:

- ✅ All fields optional
- ✅ Flexible filtering
- ✅ Range queries support (min/max)
- ✅ Clean API

MapStruct Implementation

Basic Mapper

@Mapper(componentModel = "spring") public interface VehicleMapper { /** * Convert DTO to Entity (for create/update) */ @Mapping(target = "id", ignore = true) @Mapping(target = "projectArea", ignore = true) @Mapping(target = "deleted", constant = "false") @Mapping(target = "active", constant = "true") @Mapping(target = "createdAt", expression = "java(java.time.LocalDateTime.now())") @Mapping(target = "updatedAt", expression = "java(java.time.LocalDateTime.now())") Vehicle toEntity(VehicleDTO dto); /** * Convert Entity to Response DTO */ @Mapping(source = "projectArea.id", target = "projectAreaId") @Mapping(source = "projectArea.name", target = "projectAreaName") VehicleResponseDTO toResponseDto(Vehicle entity); /** * Convert list of entities to list of response DTOs */ List<VehicleResponseDTO> toResponseDtoList(List<Vehicle> entities); /** * Update existing entity from DTO (for PATCH/PUT) */ @Mapping(target = "id", ignore = true) @Mapping(target = "projectArea", ignore = true) @Mapping(target = "deleted", ignore = true) @Mapping(target = "createdAt", ignore = true) @Mapping(target = "updatedAt", expression = "java(java.time.LocalDateTime.now())") void updateEntityFromDto(VehicleDTO dto, @MappingTarget Vehicle entity); }

Mapper Annotations

@Mapper:

- componentModel = "spring": Generate Spring bean
- uses = {OtherMapper.class}: Reuse other mappers
- injectionStrategy = InjectionStrategy.CONSTRUCTOR: Use constructor injection

@Mapping:

- source: Source field name
- target: Target field name
- ignore = true: Don’t map this field
- constant: Set constant value

- expression: Use Java expression
- qualifiedByName: Use named mapping method

@MappingTarget:

- Updates existing object instead of creating new one

Advanced Mapping Examples

1. Nested Object Mapping

@Mapper(componentModel = "spring") public interface EmployeeMapper { @Mapping(source = "projectArea.id", target = "projectAreaId") @Mapping(source = "projectArea.name", target = "projectAreaName") @Mapping(target = "age", expression = "java(calculateAge(entity.getBirthDate()))") EmployeeResponseDTO toResponseDto(Employee entity); default int calculateAge(LocalDate birthDate) { if (birthDate == null) return 0; return Period.between(birthDate, LocalDate.now()).getYears(); } }

2. Custom Mapping Methods

@Mapper(componentModel = "spring") public interface PaymentMapper { @Mapping(target = "supplier", qualifiedByName = "idToSupplier") @Mapping(target = "documents", qualifiedByName = "idsToDocuments") Payment toEntity(PaymentDTO dto); @Named("idToSupplier") default Supplier idToSupplier(Long supplierId) { if (supplierId == null) return null; Supplier supplier = new Supplier(); supplier.setId(supplierId); return supplier; } @Named("idsToDocuments") default List<TransactionalDocument> idsToDocuments(List<Long> documentIds) { if (documentIds == null) return new ArrayList<>(); return documentIds.stream() .map(id -> { TransactionalDocument doc = new TransactionalDocument(); doc.setId(id); return doc; }) .collect(Collectors.toList()); } }

3. Enum Mapping

@Mapper(componentModel = "spring") public interface VehicleMapper { // MapStruct handles enum mapping automatically by name Vehicle toEntity(VehicleDTO dto); // Custom enum mapping if needed @ValueMapping(source = "GASOLINE", target = "NAFTA") @ValueMapping(source = "DIESEL", target = "GASOIL") FuelTypeEntity mapFuelType(FuelTypeDTO fuelType); }

4. Date/Time Mapping

@Mapper(componentModel = "spring") public interface ReportMapper { @Mapping(target = "generatedAt", expression = "java(java.time.LocalDateTime.now())") @Mapping(target = "formattedDate", expression = "java(formatDate(entity.getDate()))") ReportResponseDTO toResponseDto(Report entity); default String formatDate(LocalDate date) { if (date == null) return null; return date.format(DateTimeFormatter.ofPattern("dd/MM/yyyy")); } }

Usage in Services

Service Layer Pattern

@Service @RequiredArgsConstructor public class VehicleService implements IVehicleService { private final VehicleRepository vehicleRepository; private final VehicleMapper vehicleMapper; private final ProjectAreaRepository projectAreaRepository; @Override @Transactional public VehicleResponseDTO createVehicle(VehicleDTO vehicleDTO) { // 1. Validate validateVehicle(vehicleDTO); // 2. Map DTO to Entity Vehicle vehicle = vehicleMapper.toEntity(vehicleDTO); // 3. Set relationships ProjectArea projectArea = projectAreaRepository.findById(vehicleDTO.projectAreaId()) .orElseThrow(() -> new ProjectAreaNotFoundException(vehicleDTO.projectAreaId())); vehicle.setProjectArea(projectArea); // 4. Save Vehicle savedVehicle = vehicleRepository.save(vehicle); // 5. Map Entity to Response DTO return vehicleMapper.toResponseDto(savedVehicle); } @Override @Transactional public VehicleResponseDTO updateVehicle(Long id, VehicleDTO vehicleDTO) { // 1. Find existing entity Vehicle vehicle = vehicleRepository.findByIdAndDeletedFalse(id) .orElseThrow(() -> new VehicleNotFoundException(id)); // 2. Update entity from DTO vehicleMapper.updateEntityFromDto(vehicleDTO, vehicle); // 3. Update relationships if needed if (vehicleDTO.projectAreaId() != null) { ProjectArea projectArea = projectAreaRepository.findById(vehicleDTO.projectAreaId()) .orElseThrow(() -> new ProjectAreaNotFoundException(vehicleDTO.projectAreaId())); vehicle.setProjectArea(projectArea); } // 4. Save Vehicle updatedVehicle = vehicleRepository.save(vehicle); // 5. Return response DTO return vehicleMapper.toResponseDto(updatedVehicle); } @Override @Transactional(readOnly = true) public Page<VehicleResponseDTO> getAllVehicles(VehicleFilterDTO filters, Pageable pageable) { // 1. Query with filters Page<Vehicle> vehicles = vehicleRepository.findAllWithFilters(filters, pageable); // 2. Map to response DTOs return vehicles.map(vehicleMapper::toResponseDto); } }

Controller Layer Integration

REST Controller Example

@RestController @RequestMapping("/api/v1/vehicles") @RequiredArgsConstructor @Tag(name = "Vehicles", description = "Vehicle management endpoints") public class VehicleController { private final IVehicleService vehicleService; @PostMapping @PreAuthorize("hasAuthority('VEHICLE_CREATE')") @Operation(summary = "Create new vehicle") public ResponseEntity<VehicleResponseDTO> createVehicle(@Valid @RequestBody VehicleDTO vehicleDTO) { VehicleResponseDTO response =

vehicleService.createVehicle(vehicleDTO); return ResponseEntity.status(HttpStatus.CREATED).body(response); } @GetMapping @PreAuthorize("hasAuthority('VEHICLE_READ')") @Operation(summary = "Get all vehicles with filters") public ResponseEntity<Page<VehicleResponseDTO>> getAllVehicles(@ModelAttribute VehicleFilterDTO filters, Pageable pageable) { Page<VehicleResponseDTO> vehicles = vehicleService.getAllVehicles(filters, pageable); return ResponseEntity.ok(vehicles); } @GetMapping("/{id}") @PreAuthorize("hasAuthority('VEHICLE_READ')") @Operation(summary = "Get vehicle by ID") public ResponseEntity<VehicleResponseDTO> getVehicle(@PathVariable Long id) { VehicleResponseDTO vehicle = vehicleService.getVehicleById(id); return ResponseEntity.ok(vehicle); } @PutMapping("/{id}") @PreAuthorize("hasAuthority('VEHICLE_UPDATE')") @Operation(summary = "Update vehicle") public ResponseEntity<VehicleResponseDTO> updateVehicle(@PathVariable Long id, @Valid @RequestBody VehicleDTO vehicleDTO) { VehicleResponseDTO response = vehicleService.updateVehicle(id, vehicleDTO); return ResponseEntity.ok(response); } }

Best Practices

1. DTO Naming Convention

Pattern: {Entity}{Purpose}DTO
Examples:

- VehicleDTO - Input DTO
- VehicleResponseDTO - Output DTO
- VehicleFilterDTO - Query parameters
- VehicleUpdateDTO - Partial update (if different from create)

2. Use Records for Immutability

// Good - Immutable public record UserDTO(String email, String password) {} // Avoid - Mutable (unless specifically needed) public class UserDTO { private String email; private String password; // getters and setters }

3. Separate Create and Update DTOs

When fields differ:

// Create - requires all fields public record VehicleCreateDTO(@NotNull String licensePlate, @NotNull String brand, @NotNull String model) {} // Update - all fields optional public record VehicleUpdateDTO(String licensePlate, String brand, String model) {}

4. Include Validation in DTOs

public record UserDTO(@NotBlank(message = "{user.email.required}") @Email(message =

"{user.email.invalid}") @Size(max = 100, message = "{user.email.maxLength}") String email, @NotBlank(message = "{user.password.required}") @Size(min = 8, max = 50, message = "{user.password.length}") String password) {}

5. Don’t Expose Internal IDs Unnecessarily

// Good - Minimal response public record VehicleSummaryDTO(Long id, String licensePlate, String brand, String model) {} // Avoid - Too much detail for summary public record VehicleSummaryDTO(Long id, String licensePlate, String brand, String model, LocalDateTime createdAt, LocalDateTime updatedAt, Long createdByUserId, Long modifiedByUserId // ... etc) {}

6. Use @JsonProperty for Different Field Names

public record UserResponseDTO(Long id, String email, @JsonProperty("first_name") // Snake case for API String firstName, @JsonProperty("last_name") String lastName) {}

Generated Code

MapStruct Generated Implementation

Source Mapper:

@Mapper(componentModel = "spring") public interface VehicleMapper { Vehicle toEntity(VehicleDTO dto); }
Generated Implementation:

@Component public class VehicleMapperImpl implements VehicleMapper { @Override public Vehicle toEntity(VehicleDTO dto) { if (dto == null) { return null; } Vehicle vehicle = new Vehicle(); vehicle.setLicensePlate(dto.licensePlate()); vehicle.setBrand(dto.brand()); vehicle.setModel(dto.model()); vehicle.setYear(dto.year()); vehicle.setVehicleType(dto.vehicleType()); vehicle.setFuelType(dto.fuelType()); vehicle.setTruckEquipment(dto.truckEquipment()); vehicle.setObservations(dto.observations()); return vehicle; } }
Generated files location: target/generated-sources/annotations/

Performance Considerations

1. Avoid N+1 Queries

Problem:

// Bad - N+1 query problem Page<Vehicle> vehicles = vehicleRepository.findAll(pageable); return vehicles.map(vehicle -> { // Each mapping triggers separate query for projectArea return vehicleMapper.toResponseDto(vehicle); });
Solution - Use @EntityGraph or JOIN FETCH:

@EntityGraph(attributePaths = {"projectArea"}) Page<Vehicle> vehicles = vehicleRepository.findAll(pageable);

2. Lazy Loading in DTOs

Be careful with lazy-loaded relationships:

@Transactional(readOnly = true) // Keep transaction open public VehicleResponseDTO getVehicle(Long id) { Vehicle vehicle = vehicleRepository.findById(id) .orElseThrow(() -> new VehicleNotFoundException(id)); // Mapper can access lazy-loaded relationships within transaction return vehicleMapper.toResponseDto(vehicle); }

3. Projection for Large Lists

For large datasets, use projections:

// Interface-based projection public interface VehicleSummaryProjection { Long getId(); String getLicensePlate(); String getBrand(); } // Use in repository List<VehicleSummaryProjection> findAllBy();

Testing

Unit Testing Mappers

@SpringBootTest class VehicleMapperTest { @Autowired private VehicleMapper vehicleMapper; @Test void shouldMapDtoToEntity() { // Given VehicleDTO dto = new VehicleDTO("ABC123", "Toyota", "Hilux", 2023, VehicleType.TRUCK, FuelType.DIESEL, 1L, TruckEquipment.CRANE, "Test vehicle"); // When Vehicle entity = vehicleMapper.toEntity(dto); // Then assertNotNull(entity); assertEquals(dto.licensePlate(), entity.getLicensePlate()); assertEquals(dto.brand(), entity.getBrand()); assertEquals(dto.model(), entity.getModel()); assertEquals(dto.year(), entity.getYear()); } @Test void shouldMapEntityToResponseDto() { // Given Vehicle entity = createTestVehicle(); // When VehicleResponseDTO dto = vehicleMapper.toResponseDto(entity); // Then assertNotNull(dto); assertEquals(entity.getId(), dto.id()); assertEquals(entity.getLicensePlate(), dto.licensePlate()); assertEquals(entity.getProjectArea().getId(), dto.projectAreaId()); assertEquals(entity.getProjectArea().getName(), dto.projectAreaName()); } }

Troubleshooting

Issue: Mapper not generated

Solutions:

1. Clean and rebuild: [](http://localhost:63342/markdownPreview/1531907970/markdown-preview-index-h4qbi3rq47aftn31cgn42jvml5.html#)mvn clean compile
2. Check annotation processor configuration in pom.xml
3. Ensure MapStruct version compatibility
4. Check for compilation errors

Issue: Circular dependency in mappers

Solution - Use @Context:

@Mapper(componentModel = "spring") public interface EmployeeMapper { EmployeeDTO toDto(Employee entity, @Context CycleAvoidingMappingContext context); }

Issue: Null pointer when mapping nested objects

Solution - Add null checks:

@Mapping(target = "projectAreaName", expression = "java(entity.getProjectArea() != null ? entity.getProjectArea().getName() : null)") VehicleResponseDTO toResponseDto(Vehicle entity);

Issue: Can’t find generated mapper

Solutions:

1. Check target/generated-sources/annotations/
2. Rebuild project
3. Ensure Spring component scanning includes mapper package

Migration Summary

DTOs Created: 60+
Mappers Implemented: 25+
Entities Covered: All domain entities
Pattern: Consistent across all modules
Result: Clean separation between API and domain layer ✅

Benefits

1. Type Safety

- Compile-time checking
- No runtime reflection
- IDE support

2. Performance

- No reflection overhead
- Generated code is optimized
- Fast execution

3. Maintainability

- Clear separation of concerns
- Easy to understand
- Centralized mapping logic

4. Flexibility

- Custom mapping methods
- Expression support
- Reusable mappers

5. Validation

- Bean Validation integration
- Clear validation rules
- i18n support

Conclusion

MapStruct and the DTO pattern provide a robust, type-safe, and performant solution for object mapping in the CivilControl Backend. The combination with Java Records and Bean Validation creates a clean, maintainable API layer.

Developed by: Maximo Andriola
For: Ricardo de ESEA SA
Date: January 2026
Version: 1.0
Status: Production Ready ✅